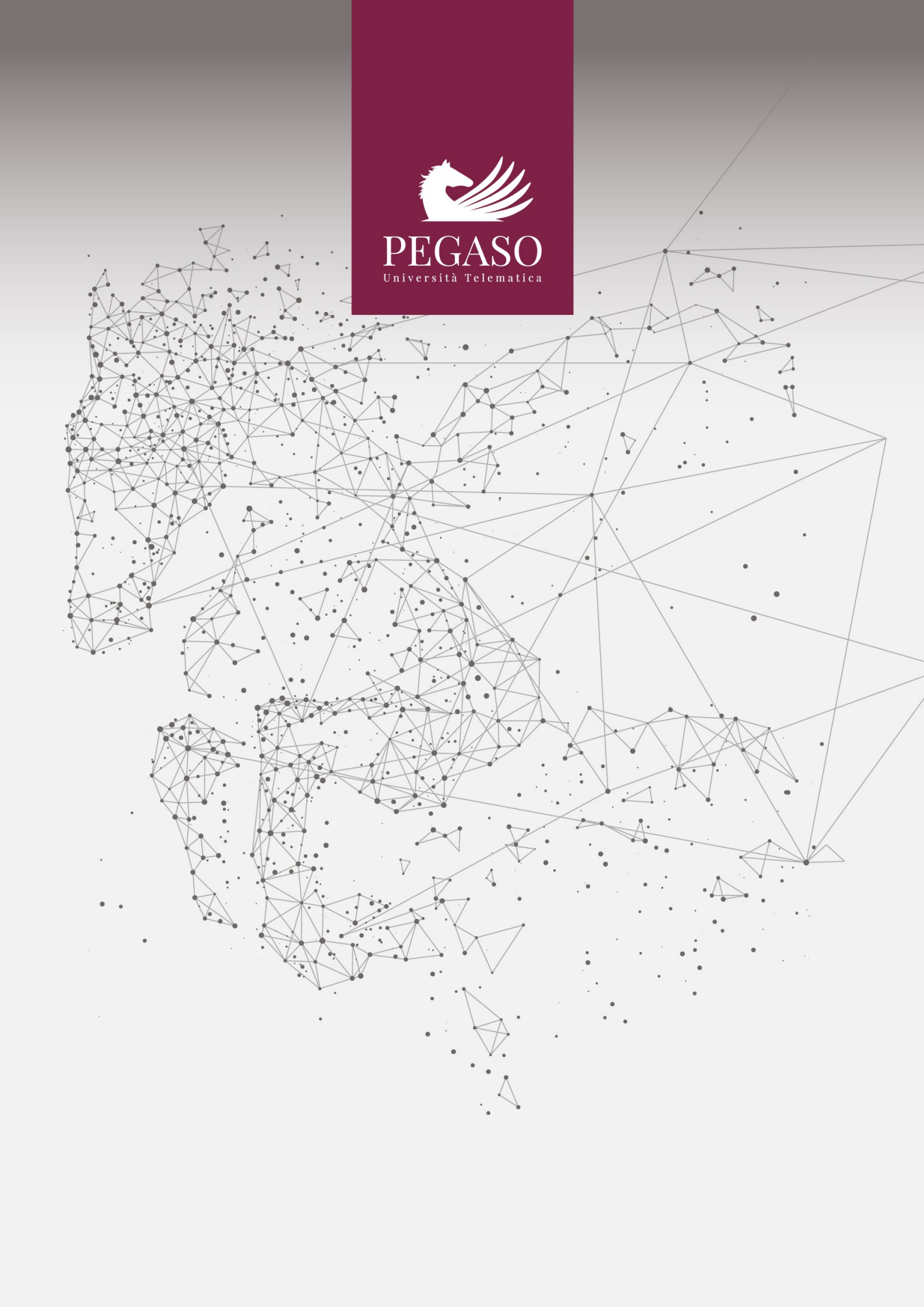

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. TIPI DI DATI COMPLESSI: ARRAY 3

2. OPERATORI 8

3. PRIORITÀ FRA GLI OPERATORI 15

BIBLIOGRAFIA 16

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Tipi di dati complessi: Array

Un array rappresenta un tipo di dato complesso che definisce una collezione di elementi di un certo tipo, indicizzati. Più precisamente, un array è una sequenza di variabili di tipo omogeneo (tutti dello stesso tipo) distinguibili l'uno dall'altro (indirizzabili) in base alla loro posizione all'intero della sequenza (indici interi).

La collezione può essere di dati primitivi o complessi o di altri array. In Fig. 1 sono mostrati degli interi che se considerati separatamente dovrebbero essere memorizzati in variabili diverse (primo, secondo, terzo, quarto, etc etc). Gli array permettono di utilizzare un solo nome per individuare una collezione costituita da vari elementi, che saranno accessibili tramite indici interi. Come mostrato in Fig.1 si può usare un array denominato numeri di 10 posizioni e riferirsi ad ognuno tramite un indice che rappresenta la posizione all'interno dell'aggregato.

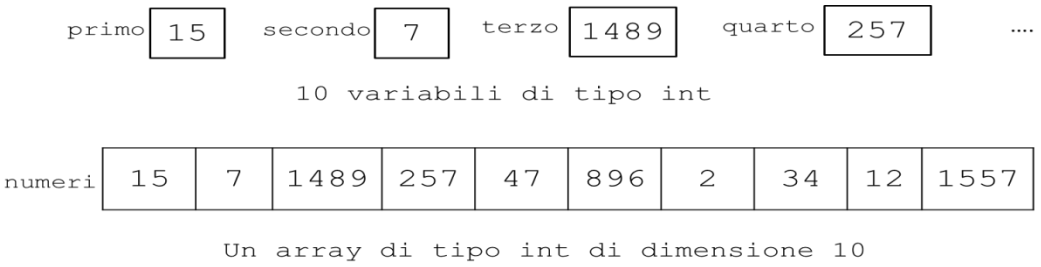


Fig. 1: Variabili e Array

In Java gli array sono, in quanto collezioni, oggetti.

Per utilizzare un array bisogna passare attraverso 3 fasi: dichiarazione, creazione ed inizializzazione (come per gli oggetti tranne per il fatto che negli oggetti si può usare il costruttore per indicare i valori iniziali).

Dichiarazione: sono possibili due modalità:

<tipo>[] <nomeArray>;

oppure

<tipo> <nomeArray> [];

In pratica per dichiarare un array è necessario posporre (oppure anteporre) una coppia di parentesi quadre all'identificatore. Le due sintassi sono perfettamente equivalenti e servono a dichiarare l'identificatore dell'array (ovvero la reference dell'array) così come avviene come primo passo per dichiarare gli oggetti in genere. Dopo questa operazione, come mostrato in Fig. 2, si avrà ad esempio l'identificatore numeri che è una reference ad un array di interi, l'identificatore prezzi che è una reference

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

ad un array di double, l'identificatore nomi che è una reference ad un array di String (oggetti per memorizzare e trattare stringhe di caratteri).

```
…
int [] numeri ;      // dichiara un array di interi
double [] prezzi ;   // dichiara un array di numeri frazionari
String [] nomi ;     // dichiara un array di stringhe
…
```

```
…
int numeri [];       // dichiara un array di interi
double prezzi [];    // dichiara un array di numeri frazionari
String nomi [];      // dichiara un array di stringhe
…
```

Fig. 2: Esempi di dichiarazioni di array

Con le istruzioni precedenti abbiamo dichiarato l'identificatore dell'array (ovvero la reference dell'array) dove non è necessario specificare i termini (ovvero la lunghezza dell'array) all'interno delle parentesi quadre. Un array è un oggetto a tutti gli effetti e come avviene per tutti gli oggetti, occorre adesso procedere alla creazione di una sua istanza tramite una new, ma con una sintassi speciale.

Creazione: la sintassi è:

<nomeArray> = new <tipo>[num_elementi]

Come si può notare è obbligatorio specificare, al momento della creazione di un'istanza dell'array, la dimensione dell'array stesso tra le parentesi quadre. Questa dimensione, una volta dichiarata, non è più modificabile. Essa infatti corrisponde all'allocazione di un'area di memoria idonea per ospitare i valori corrispondenti e l'area di memoria una volta riservata all'array tale rimane e inalterata nella sua dimensione. Dichiarazione e creazione possono essere abbinati.

```
…
numeri = new int [10];      // crea un array di 10 interi
prezzi = new double [25];   // crea un array di 25 numeri fraz.
nomi = new String [100];    // crea un array di 100 riferimenti
                            // a stringhe
…
```

```
…
int [] numeri = new int [10];
…
```

Fig. 3: Esempi di creazione di array

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La creazione degli array è mostrata in Fig. 3. In entrami i casi si è proceduto tramite la new ad abbinare alla reference un'area di memoria in cui ospitare l'array. Nel caso dell'array di interi (double) si è allocato un'area di memoria riservata a memorizzare 10 celle di 4 byte (8 byte) ciascuna inizializzate al valore di default per gli interi che è 0. Nel caso dell'array di String (o di oggetti in generale) si è allocato un'area di memoria per memorizzare i reference (uno per ogni posizione dell'array) ovvero i valori degli indirizzi che punteranno agli oggetti, ma non si sono istanziati tali oggetti (nel caso specifico di tipo String). Tali valori di indirizzi sono inizializzati ad un valore di default che per gli array di oggetti è null.

Inizializzazione: occorre inizializzare ogni elemento singolarmente attraverso assegnazione dei valori. Si noti la differenze tra un array dichiarato di tipo di dato primitivo o complesso (vedi Fig. 4). Il primo contiene direttamente i suoi elementi. Il secondo contiene solo reference, non gli elementi stessi e quindi per occorre inizializzare questi (fino a questo momento a null) attraverso una new sul tipo dell'oggetto. Adesso l'array di String punta ad un'area di memoria allocata e inizializzata attraverso apposito metodo costruttore con in input la stringa specifica dei nomi.

```
…
// inizializzazione array numeri di 10 interi
numeri[0] = 15;
numeri[1] = 7;
…
numeri[9] = 1557;
…
// inizializzazione array nomi di 100 stringhe
nomi[0] = new String ("Anna");
nomi[1] = new String ("Carlo");
nomi[2] = new String ("Maria");
…
nomi[99] = new String ("Alfonso");
```

Fig. 4: Esempi di inizializzazione di array

L'indice di un array inizia sempre da zero e arriva a numero di posizioni dell'array meno uno.

Il fatto che ogni elemento dell'array debba essere dichiarato singolarmente non significa che occorra dedicare un'istruzione di codice specifica per ogni elemento come abbiamo visto in questo caso. Vedremo come rendere questa operazione più agile nel codice sfruttando l'indice dell'array all'interno dei cicli. Essi ci permettono di assegnare i valori di un array in maniera molto più compatti.

Come mostrato in Fig. 5, esiste anche un modo più compatto per eseguire i 3 passi visti per dichiarare, istanziare e inizializzare un array in un passo solo tramite una sintassi breve in due versioni, con e senza uso esplicito di una new.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
…
// dichiarazione, creazione e inizializzazione array numeri di
//10 interi

int numeri[] =  {15,7,1489,257,47,896,2,34,12,1557};
int numeri[] = new int [] {15,7,1489,257,47,896,2,34,12,1557};

// dichiarazione, creazione e inizializzazione array nomi di
//100 stringhe

String nomi [] =  {new String ("Anna"),…};
// una new per ogni elemento dell'array nomi
String nomi [] =  new String[]{new String ("Anna"),…};
// una new per ogni elemento dell'array nomi
```

Fig. 5: Sintassi compatta per dichiarazione, creazione e inizializzazione di array

In quanto oggetti, gli array definiscono al loro interno delle variabili e dei metodi implicitamente. Per esempio essi definiscono come attributo una costante pubblica chiamata lenght che restituisce la dimensione effettiva dell'array stesso definita in fase di dichiarazione. Ricordiamo che la dimensione di un array una volta impostata non può essere modificata. Questo può essere utile in quanto non sempre si conosce a priori la dimensione di un array (si veda esempio in Fig. 6). Attenzione: length non è un metodo (non è seguito da parentesi).

```
…
int [] numeri = new int [10];
System.out.println (numeri.length) // stampa 10
…


....
if (grande)
numeri = new int [1000];
else
numeri = new int [10];
.....
// qual e' la lunghezza di numeri ?
```

Fig. 6: Uso dell'attributo lenght di array

a. Array multidimensionali

Esistono anche array bidimensionali che sono array di array. Gli array possono anche essere tridimensionali e in generale multidimensionali, anche a dieci dimensioni. A differenza della maggior parte

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

degli altri linguaggi di programmazione, in Java un array bidimensionale non deve per forza essere rettangolare. Fin Fig. 7 è mostrato un esempio di dichiarazione, creazione e inizializzazione.

```
…
// dichiarazione e creazione di un array bidimensionle di
//interi
int arrayNonRettangolare [][] = new int [4][];
arrayNonRettangolare[0] = new int[2];
arrayNonRettangolare[1] = new int[4];
arrayNonRettangolare[2] = new int[8];
arrayNonRettangolare[3] = new int[8];
//inizializzazione
arrayNonRettangolare[0][0] = 1;
arrayNonRettangolare[0][1] = 2;
arrayNonRettangolare[1][0] = 1;
…
arrayNonRettangolare[3][7] = 10;
```

Fig. 7: Array multidimensionale

Dapprima si dichiara e si crea un array bidimensionale. Si specifica solo la prima dimensione fatta di 4 elementi. Ogni elemento di questi sarà a sua volta un altro array di lunghezza ancora da scegliere. Questo viene specificato tramite l'allocazione dei 4 elementi e con la relativa new dell'elemento. Per il primo (indice 0) si tratta di un altro array di interi a 2 posizioni, ciascuno poi inizializzato a 1 e 2. Poi a seguire le altre inizializzazioni.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Operatori

In questa sezione andremo a presentare la lista degli operatori di base che Java mette a disposizione. Ci concentreremo su quelli principali rappresentati con simboli (ad esempio anche new e altri similari in Java sono considerti operatori ma non sono considerati adesso). Gli operatori si classificano in unari (ovvero che fanno operazioni con un solo operando) e binari (ovvero fanno operazioni con due operandi). Esiste anche un operatore ternario detto anche operatore condizionale che può regolare il flusso di esecuzione con una condizione. Vedremo i seguenti tipi di operatori: numerici, di assegnamento, relazionali o di confronto, logico-booleani, di concatenamento di stringhe.

a. Operatore di assegnamento

L'operatore di assegnamento = è un operatore binario, il cui primo operando deve essere una variabile. Il secondo operando può essere un valore compatibile con il tipo della variabile, oppure un'altra variabile. E' possibile assengare valori diversi ad una variabile durante l'esecuzione del codice, a meno che essa non sia dichiarata final. In Fig. 8 sono mostrate tutte assegnazioni valide. Dopo queste istruzioni il valore intero 2 sarà immagazzinato sia in var1 e var2. Infatti, con l'ultima istruzione sarà assegnata una copia del valore del secondo operando (var2) al primo operando (var1).

```
…
int var1 = 0;
int var1 = 1;
int var2 = 2;
var1 = var2;   //il valore 2 sarà immagazzinato in var1 e var2
…
```

Fig. 8: Uso dell'operatore di assegnamento

b. Operatore numerici

Qui sono mostrati gli operatori numerici di cui si è tutti già familiari a prescindere dal linguaggio e dalla programmazione. L'unico operatore che può risultatre non famiare è l'operatore modulo. Il risultato dell'operazione modulo fra due numeri coincide con il resto della divisione fra essi. Ad esempio 5 % 3 restituisce 2.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il motivo per cui sono esplicitamente mostrati in Fig. 9 gli operatori numerici è perché esistono in Java esistono anche gli operatori di assegnazione composti come spiegato nel paragrafo successivo.

Descrizione	Operatore
Somma	+
Sottrazione	-
Moltiplicazione	*
Divisione	/
Modulo	%

Fig. 9: Operatori numerici

c. Operatore di assegnamento composti

Gli operatori di assegnamento composti oltre a svolgere un'operazione, assegnano anche il valore del risultato al primo operando, rendendo il codice meno verboso. Per esempio, come mostrato in Fig. 10, se abbiamo: int i=5;scrivere i=i+2; è equivalente a scrivere i +=2;

Descrizione	Operatore	Esempio
Somma ed assegnazione	+=	i+=2 (i=i+2)
Sottrazione ed assegnazione	-=	i-=2 (i=i-2)
Moltiplicazione ed assegnazione	*=	i*=2 (i=i*2)
Divisione ed assegnazione	/=	i/=2 (i=i/2)
Modulo ed assegnazione	%=	i%=2 (i=i%2)
Pre-incremento di un'unità	++	++i (i+=1)
Pre-decremento di un'unità	--	--i (i-=1)
Post-incremento di un'unità	++	i++ (i+=1)
Post-decremento di un'unità	--	i-- (i-=1)

Fig. 10: Operatori di assegnamento composti

Poi ci sono gli operatori di pre- e post-incremento e pre- e post-decremento. Si tratti di operatori unari che possono essere usati per incrementare o decrementare di una sola unità una variabile numerica.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Ad esempio, se vogliamo incrementare di una sola unità una variabile numerica possiamo scrivere i=i+1; oppure i+=1; ma anche i++; oppure ++i; ottenendo lo stesso risultato ovvero il valore della variabile i è incrementato di 1 e assegnato nuovamente alla variabile stessa.

La differenza tra pre-incremento o post-incremento consiste essenzialmente nelle priorità che essi hanno rispetto all'operatore di assegnazione =. L'operatore di post-incremento ha minore priorità rispetto all'operatore di assegnazione =. Tale differenza si percepisce solo quando tali operatori sono presenti nello stesso statement dove è presente un'assegnazione. Come mostrato in Fig. 11, con x=5; y=++x; si avrà x=6; y=6; Infatti nella seconda istruzione verrà prima eseguita l'operazione di pre-incremento che cambierà il valore della variabile x, poi tale valore verrà assegnato a y. Nel caso di x=5; y=x++; il risultato sarà x=6 y=5. Infatti l'operatore di assegnazione ha maggiore priorità rispetto all'operatore di post-incremento, quindi nella seconda istruzione sarà prima assegnato il valore della variabile x ad y (ovvero 5) e poi il valore di x verrà incremento a 6.

```
…
//per incremenare di 1
int i = 5;
i=i+1; // è equivamente a
i+=1;  // è equivamente a
i++;   // è equivamente a
++i;   // La differenza è nella priorità che hanno
       // rispetto all'operatore di assegnazione =
…
int x,y;
x=5;
y=++x; // si avrà x=6 e y=6
…
x=5;
y=x++; // si avrà x=6 e y=5
```

Fig. 11: Uso operatori di pre- e post- incremento

d. Operatori relazionali o di confronto

Gli operatori relazionali sono operatori binari il cui risultato è sempre un valore booleano, ovvero true o false. La tabella in Fig. 12 riporta tutti gli operatori appartenenti a questa categoria fondamentale.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Operatore	Simbolo	Applicabilità
Uguale a	==	Tutti i tipi
Diverso da	!=	Tutti i tipi
Maggiore	>	Solo i tipi numerici
Minore	<	Solo i tipi numerici
Maggiore o uguale	>=	Solo i tipi numerici
Minore o uguale	<=	Solo i tipi numerici

Fig. 12: Operatori relazionali

Il loro funzionamento è molto intuitivo e essenzialmente noto, può essere insidiosa la situazione di combinazione con altri operatori. Come mostrato negli esempi in Fig. XX, la variabile boolean b è assegnata con il risultato dell'operazione di confronto (9==9) ovvero true. Questa operazione contiene due operatori, un operatore di assegnazione = ed uno di confronto ==. In questo caso le parentesi servono a migliorare la leggibilità del codice, in realtà sono superflue in quanto l'operatore di assegnazione ha sempre minore probabilità rispetto agli altri operatori, quindi verrà eseguito per ultimo.

Nel secondo esempio la variabile b1 vale false, perché i non è diverso da d (hanno lo stesso valore nonostante siano di tipo diverso). Quindi (i!=d) vale false, e questo valore viene assegnato a b1. Aggiungiamo un'istruzione leggermente più complicata contenente due operatori di confronto (eseguiti per primi) e uno di assegnazione. In questo caso b2 vale true perché i<=11 è falso come lo è b1. E quindi dal loro confronto false == false risulta il valore di true.

```
…
boolean b = (9 == 9); // b vale true;
…
int i = 10;
double d = 10.0;
boolean b1 = (i!=d); // b1 vale falso;
boolean b2 = b1 == (i>=11); // b2 vale true;
…
```

Fig. 12: Uso operatori relazionali

Se confrontiamo due reference con l'operatore == il risultato sarà true se e solo se i due reference puntano allo stesso oggetto altrimenti sarà false. Infatti, viene confrontato sempre il valore delle variabili in gioco, e il valore di una variabile reference è l'indirizzo della zona di memoria con l'oggetto è memorizzato.

Nell'esempio di Fig. 13, nel primo caso stamperà false. Infatti, il confronto booleano verrà fatto sull'indirizzo dei due oggetti creati, che puntano evidentemente a due oggetti diversi (anche se contengono

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

lo stesso valore per l'attributo nome). Nel secondo caso i due reference hanno lo stesso valore (alunno2 prende il valore di alunno1) andando a puntare lo stesso oggetto. In questo caso ovviamente il risultato del confronto è true.

```
…
public class Alunno {
        public String nome;
        public Alunno (String n) { nome = n;}
}
…
Alunno alunno1 = new Alunno ("Simone");
Alunno alunno2 = new Alunno ("Simone");
System.out.println(alunno1==alunno2); // stamperà false
…
Alunno alunno1 = new Alunno ("Simone");
Alunno alunno2 = alunno1;
System.out.println(alunno1==alunno2); // stamperà true
```

Fig. 13: Uso di operatori relazionali tra reference

e. Operatori logico-booleani

Gli operatori logico-booleani usano solo operandi di tipo booleano con risultato ovviamente di tipo booleano. Il NOT logico è l'unico operatore unario tra gli operatori logici e si usa anteponendolo al suo operando. Le versioni short circuit hanno la caratteristica di non valutare il secondo operando nel caso il primo sia sufficiente a determinare il risultato dell'operazione. Questo significa che se in un'operazione di AND con corto circuito il primo operatore è false non veràà valutato il secondo operando. Infatti, qualsiasi sia il suo valore, il risultato dell'operatore sarà false. Allo stesso modo, se in un'operazione di OR con corto circuito il primo operando risulta true, non sarà valutato il secondo operando perché sarebbe superfluo, in quanto il risultato il risultato dell'operazione sarà comunque true. La lista è fornita in Fig. 14.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Descrizione	Operatore
NOT logico	!
AND logico	&
OR logico	|
XOR logico	^
Short circuit AND	&&
Short circuit OR	||
AND e assegnazione	&=
OR e assegnazione	|=
XOR e assegnazione	^=

Fig. 14: Operatori logico-booleani

Ricapitolando un'espressione booleana (un'espressione che porta ad un risultato true o false) può consistere dei seguenti elementi facendo anche uso degli operatori relazionali e/o logico-booleani:

- Lettarali: in generale i letterali sono valori costanti che si assegnano alle variabili. Tra i literal sono inclusi true e false come lo sono naturalmente (anche se può sembrare scontato) i numeri o i caratteri (tra apici) «w», «\n». In questo contesto sono naturalmente nominati true e false perché più frequentemente possono apparire in un'espressione booleana, ma anche i numeri o caratteri possono apparire in operazioni di confronto.
- Variabili di tipo booleano: usate nelle operazioni di confronto e/o logico-booleane. Ad esempio trovato è una variabile booleana che può essere confrontato
- Confronti fra variabli e/o literali più o meno complesse (x%2)==0; oppure ((lordo-netto)/lordo)<(sconto_max/lordo)
- Operazioni logiche tra espressioni booleane

f. Operatori di concatenamento di stringhe

Il simbolo +, oltre ad essere l'operatore aritmetico di addizione, rappresenta anche l'operatore per concatenare le stringhe. Come mostrato in Fig. XX, , il seguente frammento di codice:

```
String nome = ”Giacomo";
String cognome = ”Sabatini";
String nomeCompleto = ”Sig. " + nome + ”" + cognome;
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

farà in modo che la stringa nomeCompleto abbia Sig. Giacomo Sabatini come valore. Se sommiamo un qualsiasi tipo di dato con una stringa, il tipo di dato sarà automaticamente convertito in stringa, risultando spesso molto utile.

Bisogna però stare attenti ad alcune situazioni. Per esempio:

System.out.println (1+1+"3" + 7);

stamperà 237. Infatti priva viene eseguita la somma fra due interi 1+1 che ritorna 2, ma quest'ultimo valore viene sommato alla stringa «3», e quindi viene tramutato anch'esso nella stringa «23». Anche l'ultimo operatore è da considerarsi un operatore di concatenazione. Infatti somma la stringa «23» all'intero 7 che viene tramutato nella stringa finale «237».

```
…
String nome = "Giacomo";
String cognome = "Sabatini";
String nomeCompleto = «Sig. " + nome + "" + cognome;
                                    //varrà Sig. Giacomo Sabatini
…
//attenzione a
System.out.println (1+1+"3" + 7); //stamperà 237!
```

Fig. 15: Uso dell'operatore di concatenamento fra stringhe

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Priorità fra gli operatori

Nella tabella di Fig. 16 sono riportati in ordine di priorità tutti gli operatori di Java (non tutti trattati). Si può consultare questa tabella al bisogno senza necessità di impararla a memoria. E' tuttavia importante sapere che esistono delle priorità tra gli operatori per usarli in modo appropriato e non incorrere in errori inconsapevoli anche difficile da scovare. Oppure nell'incertezza si può usare le parentesi tonde così come faremmo nell'aritmetica tradizionale, talvolta comunque necessarie.

Descrizione	Operatore
da sx a dx	++ -- + - ~ !
da sx a dx	* / %
da sx a dx	+ -
da sx a dx	<< >> >>>
da sx a dx	== !=
da sx a dx	= *= /= %= -= <<= >>= >>>= &= ^= |=

Fig. 16: Priorità tra gli operatori

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- [1] Il nuovo Java, Claudio De Sio Cesari, Hoepli Informatica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).